



Unit 1 — Basic concepts of Artificial Intelligence

CHAPTER 1.5 • DATA REPRESENTATION & PROGRAMMING

Data representation and programming in Python

Datatypes • variables • operators • conditional statements
control statements • functions

Robotics and AI Dept



Dr. Saptarshi Jana
saptarshi.jana@techvein.com

PYTHON

2

Introduction

Why python?

A beginner-friendly programming language



Key Features and Advantages

Easy-to-learn, Dynamic Typing, Portability, High-level-language,

community support, Open-Source, Huge Library support



Applications

Data Analysis and
Visualization

Web Development

Machine Learning and AI

Scientific Computing

Game Development



Syntax:

Syntax refers to the rules that define the structure of valid statements

Variables

Comments

Indentation



Flow Chart

A type of diagram that represents an algorithm, workflow or process.

COMPARISON

C++

```
#include <iostream> // Required for input/output operations

int main() {
    int myNumber = 123; // Declare an integer variable and as

    // Print the value of the variable to the console
    std::cout << "The number is: " << myNumber << std::endl;

    return 0; // Indicate successful program execution
}
```

Java

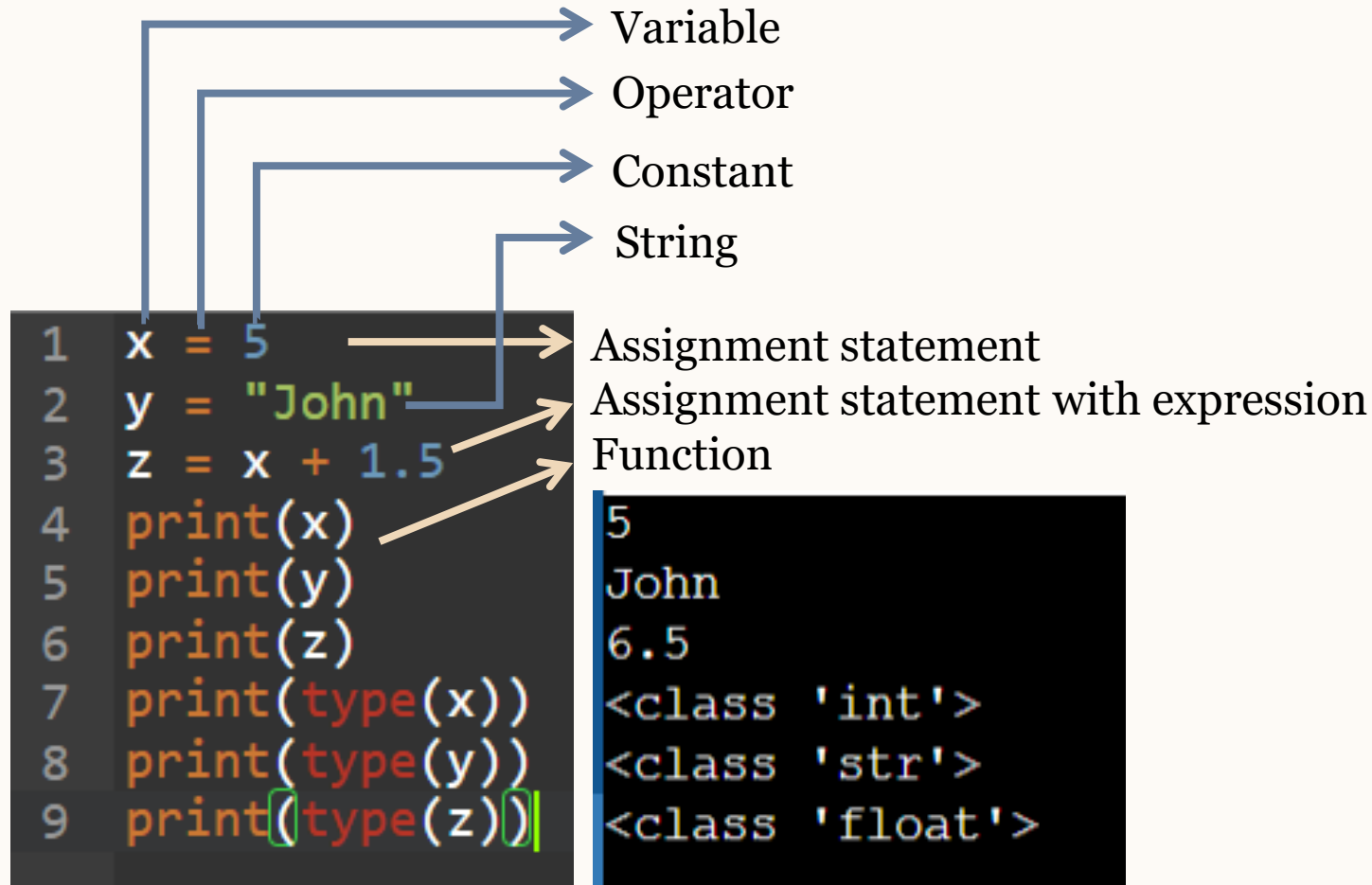
```
public class PrintNumber {
    public static void main(String[] args) {
        // Declare and initialize an integer variable
        int myNumber = 123;

        // Print the value of the variable
        System.out.println(myNumber);
    }
}
```

Python

```
print(123)
```

LINES



BASIC SYNTAX OF PYTHON

Variables:

- Think of variables as containers for storing data.

Example:

```
x = 10 # stores the number 10 in x.
```

Indentation:

- Indentation defines code structure. Like organizing paragraphs in an essay.

Example:

```
if x > 0:  
    print("Positive number")
```

Comments:

- Single-line comments start with #.
- Multi-line comments use triple quotes (""").

VARIABLES

Names	<pre>myvar = "John" my_var = "John" _my_var = 'John'</pre>	<pre>2myvar = "John" my-var = "John" my var = "John"</pre>
Input	<pre>input()</pre>	<pre>x = input("Enter a number") Print(x)</pre>
Multiple	<pre>x, y, z = "Orange", "Banana", "Cherry"</pre>	<pre>x = y = z = "Orange"</pre>
Output	<pre>x = "Python " y = "is " z = "awesome" print(x + y + z)</pre>	<pre>x = 5 y = "John" print(x, y)</pre>
Global	<pre>x = "awesome" def myfunc(): x = "fantastic" print("Python is " + x) ...</pre>	<pre>myfunc() print("Python is " + x)</pre>

RESERVED WORDS OF PYTHON

The first thing we have in every programming language is reserved words.

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

DATA TYPES

		Data Type	Description	Example
		Numeric	Represents numbers	int (e.g., 5), float (e.g., 3.14), complex (e.g., 2 + 3j)
		Boolean	Represents truth values	bool (e.g., True, False)
Text Type		String	Represents text	str (e.g., "Hello, world!")
Collection Data Types	Sequence Types	List	Ordered, mutable collection	list (e.g., [1, 2, "three"])
		Tuple	Ordered, immutable collection	tuple (e.g., (1, 2, "three"))
		Set	Unordered collection of unique elements	set (e.g., {1, 2, 3}), frozenset(e.g., frozenset {1, 2, 3})
	Mapping Type	Dictionary	Unordered collection of key-value pairs	dict (e.g., {"name": "Alice", "age":
		NoneType	Absence of any value	x = None

TYPE CASTING

- Convert between types using `int()`, `float()`, `str()`, etc.

Casting	<code>z = float(3) # z will be 3.0</code>	<code>x = str(3) # x will be '3'</code>
Explicit Type Conversion	<code>x = input("Enter a number") Print(type(x))</code>	<code>x = int(input("Enter a number")) Print(type(x))</code>
Implicit Type Conversion	<code>x = 2 # Check type y = 2.5 # Check type</code>	<code>z = x + y # Check type</code>

TASK:

Simple Interest: Calculate the simple interest using the formula: $\text{Simple Interest} = P \cdot R \cdot T / 100$

Input:

Principal (P), rate of interest (R), and time (T) in years.

Output:

The simple interest.

Swap Two Numbers: Swap the values of two variables.

Input:

Two numbers.

Output:

The numbers after swapping.

Script:

NUMBERS

```
x = 1      # int
y = 2.8    # float
z = 1j     # complex
```

```
#convert from int to float:
```

```
a = float(x)
```

```
#convert from float to int:
```

```
b = int(y)
```

```
#convert from int to complex:
```

```
c = complex(x)
```

```
# range(1, 10)
```

```
#Import the random module, and display a random  
number from 1 to 9:
```

```
import random
print(random.randrange(1, 10))
```

STRING

```
a = "Hello, World!"
print(a)
print(a[1])
print(len(a))
```

```
# characters by using the slice syntax
```

```
print(a[2:5])
print(a[2:])
print(a[:5])
print(a[-5:-2])
print(a.upper()) # uppercase
print(a.strip()) # removes any space at beginning,
print(a.replace("H", "J"))
print(a.split(",")) # returns ['Hello', ' World!']
```

```
# Looping through Strings:
```

```
for x in "banana":
    print(x)
```

```
age = 25
txt = f"My name is Xavi, I am {age}"
print(txt)
```

OPERATOR TYPE

Operator Type	Operator	Description	Example
Arithmetic Operators	+, -, *, /, %, **, //	Perform basic arithmetic operations	5 + 3 = 8, 10 % 3 = 1
Assignment Operators	=, +=, -=, *=, /=, %=, //=, **=, &=, `	=, ^=, >>=, <<=	Assign and modify variable values
Comparison Operators	==, !=, >, <, >=, <=	Compare values and return True or False	5 > 3 (True), 5 == 3 (False)
Logical Operators	and, or, not	Combine conditional statements	(5 > 3 and 2 < 4) (True)
Identity Operators	is, is not	Check if two objects are the same (by memory location)	x is y, x is not y
Membership Operators	in, not in	Check for membership in a sequence (list, tuple, etc.)	'a' in 'apple' (True)

TASK

Write a program that takes a string as input and checks if it's a palindrome (a word that reads the same backwards and forward). Ignore spaces and cases.

```
s = input("Enter a string that you want to check: ")
s_lower = s.lower() # modifies string into lower case
s_no_space_lower = s_lower.replace(" ", "")
#removes all spaces from the string
j = s_no_space_lower[::-1]
# negative indexes to start from the end of the
string
print("Is <"+ s +"> palindrome?",s_no_space_lower==j)
# True or False
```

Control Statements

- ❖ Sequential Pattern
- ❖ Conditional Pattern
- ❖ Repeated Patterns

IF-ELSE CONDITION

- **Definition:**

- Conditional statements allow a program to make decisions based on conditions.

- **Syntax:**

```
if condition:
```

```
    # Code block for true condition
```

```
else:
```

```
    # Code block for false condition
```

- **Example:**

```
num = 10
```

```
if num > 0:
```

```
    print('Positive number')
```

```
else:
```

```
    print('Non-positive number')
```

FOR LOOP

- **Definition:**

- Used to iterate over a sequence (like a list, tuple, or string).

- **Syntax:**

for variable in sequence:

 # Code block for each item in sequence

- **Example:**

for i in range(5):

 print(i) # Prints numbers from 0 to 4

- **Use Case:** Iterate over items, repeat actions a specific number of times.

WHILE LOOP

- **Definition:**

- Repeats as long as the condition is ``True``.

- **Syntax:**

```
while condition:
```

```
    # Code block to execute while condition is True
```

- **Example:**

```
count = 0
```

```
while count < 5:
```

```
    print(count)
```

```
    count += 1
```

- **Use Case:** Perform tasks until a condition changes

CONTROL FLOW STATEMENTS

- **break:** The break statement immediately terminates the loop
- **continue:** The continue statement skips the rest of the code within the current iteration.
- **pass:** The pass statement is a null operation; it does nothing.

TASK:

Write a Python program to compare three numbers (by taking as an input) and find the greatest among them.

Script: (Hint)

```
num1 = ??(?("Enter the first number: "))  
num2 = ??(?("Enter the second number: "))  
num3 = ??(?("Enter the third number: "))
```

```
if <cond> and <cond>:  
    greatest = ?  
elif <cond> :  
    greatest = ?  
else:  
    greatest = ?
```

```
Print(?)
```

COLLECTION DATA TYPES

- ❖ LIST
- ❖ TUPLE
- ❖ SET
- ❖ DICTIONARY

LIST, TUPLE AND SET

Names	List [10, "item", True]	Tuple (10, "item", True)	Set {10, "item", True}
Order	Maintains the order of elements	Yes	No
Mutable	Can be modified (add, remove, or update elements)	No	Elements can be added or removed only
Duplicates	Multiple identical items can exist	Yes	No
Speciality	Supports different data types in the same list	Faster than lists due to immutability	Supports mathematical set operations

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered (3.7 onwards) and changeable. No duplicate members.

LIST, TUPLE AND SET



› list_1 = ["apple", "banana", "cherry", "dates"]



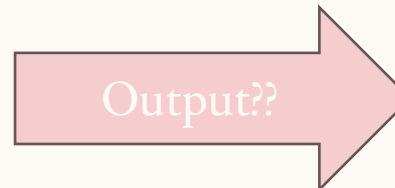
› tuple_1 = ("string", 34, True, 40, "male")

› set_1 = {1, 2, 3, 2}

› print(type(list_1))

› print(type(tuple_1))

› print(type(set_1))



Note: The first item has index 0

DICTIONARY

A dictionary in Python is an unordered, mutable collection of key-value pairs.

- **Keys:**

Keys must be unique and immutable (e.g., numbers, strings, tuples). They serve as identifiers for accessing the associated values.

- **Values:**

Values can be of any data type and can be duplicated.

```
dict = {  
    "brand": "St. Xavier's",  
    "model": "Panihati",  
    "estd_year": 1967,  
    "affiliation": "CISCE"  
}  
print(dict)
```

WHY USE FUNCTIONS?

- Makes code reusable.
- Simplifies complex programs.
- Makes debugging easier.
- Helps in better organization of code.

WHAT IS A FUNCTION?

- A function is a block of reusable code.
- It performs a specific task.
- Functions help avoid repetition and make programs modular.

TYPES OF FUNCTIONS

1. Built-in functions: Provided by Python (e.g., ``len()``, ``print()``).
2. User-defined functions: Functions created by programmers.

DEFINING A FUNCTION

USE THE `DEF` KEYWORD TO DEFINE A FUNCTION

Syntax:

```
def function_name(parameters):  
    # Code block  
    return value
```

The diagram illustrates the execution flow of the provided code. A blue box labeled '1' encloses the function definition and the function call. A blue arrow labeled '2' points down to the function definition. A blue arrow labeled '3' points from the function call to the function definition, indicating the call process.

```
def greet():  
    print('Hello World!')  
  
# call the function  
greet()  
  
print('Outside function')
```

EXAMPLE

```
def greet(name, msg):  
    print(f"{msg}, {name}")  
  
greet(name="Class 9", msg="Welcome") # Keyword arguments  
  
greet("Class 9", "Welcome") # positional arguments  
  
greet() # positional arguments
```

```
def greet(name="Guest"):  
    print(f"Hello, {name}!")  
  
greet("Alice") # Positional arguments  
greet()       # Default arguments
```

TASK

Write a Python program to compare three numbers (taken as input) and determine the greatest among them.

```
def find_greatest(a, b, c):  
    return max(a, b, c) # max() is a built-in  
function for comparing numbers and finding out  
the maximum among those.  
# Example usage  
print(find_greatest(3, 7, 5))
```



THANK YOU

Dr. Saptarshi Jana
Robotics and AI Faculty
Saptarshi.jana@techvein.com